Содержание

Введение 3

1 Теоретическая часть 5

1.1 Описание предметной области 5

1.2 Анализ существующей ситуации 6

1.3 Постановка задачи 7

1.4 Анализ существующих разработок и обоснование актуальности нового проекта 9

2 Специальная часть 11

2.1 Выбор технологий и инструментальных средств 11

2.1.1 Выбор подхода к разработке 11

2.1.2 Выбор среды разработки и языка программирования 12

2.2 Разработка спецификаций 15

2.2.1 Разработка диаграмм вариантов использования 15

2.2.2 Разработка диаграмм последовательностей системы 15

2.2.3 Разработка диаграммы пакетов 16

2.2.4 Разработка диаграммы классов 18

2.2.5 Разработка диаграммы состояний 18

2.2.6 Разработка диаграммы деятельности 20

2.2.8 Разработка инфологической модели базы данных 21

2.2.9 Разработка физической модели базы данных 21

2.2.10 Разработка диаграммы размещения 22
2.3 Проектирование программного модуля 22
2.3.1 Разработка алгоритмов реализации основных функций программного обеспечения 23
2.3.2 Проектирование пользовательского интерфейса 23
2.4 Реализация программного обеспечения на выбранном языке программирования и в выбранной среде разработки 23
2.5 Выбор стратегии тестирования, разработка тестов, тестирование и отладка программного обеспечения 23
2.6 Разработка эксплуатационной документации 23
2.6.1 Разработка руководства системного программиста 23
2.6.2 Разработка руководства пользователя 23
3 Экономическая часть 24
3.1 Расчет трудоемкости разработки программного продукта 27
3.2. Составление сметы затрат на разработку 31
3.3. Анализ экономической эффективности при внедрении программного обеспечения 35
Охрана труда 39
Заключение 51
Список используемых источников 52
Графическая часть 53
Техническое задание 54
Руководство системного программиста 66
Руководство пользователя 67
Фрагмент листинга программы 68

Введение

Современный этап развития информационных технологий характеризуется активным формированием цифровых экосистем, объединяющих множество сервисов в рамках единой технологической платформы. Крупные мировые компании, такие как Google, Яндекс и Microsoft, строят свои продукты на основе распределённых архитектур, обеспечивающих масштабируемость, отказоустойчивость и централизованное управление доступом пользователей. Ключевым элементом подобных решений является ядро цифровой экосистемы, реализованное на основе микросервисной архитектуры с единой системой аутентификации и API-шлюзом.

В условиях роста количества онлайн-сервисов и увеличения числа пользователей особую значимость приобретает обеспечение безопасного и унифицированного взаимодействия между компонентами системы. Отсутствие централизованной аутентификации и единой точки входа приводит к дублированию функциональности, усложнению сопровождения, снижению уровня безопасности и ограничению возможностей масштабирования. Разрозненные сервисы без общей архитектурной основы затрудняют интеграцию новых компонентов и увеличивают нагрузку на разработчиков и администраторов инфраструктуры.

Разработка ядра цифровой экосистемы на основе микросервисной архитектуры позволяет создать гибкую и расширяемую платформу, обеспечивающую централизованное управление пользователями, маршрутизацию запросов через API-шлюз, контроль доступа и безопасное межсервисное взаимодействие. Такое решение формирует основу для построения различных цифровых сервисов, которые могут развиваться независимо друг от друга, сохраняя при этом целостность общей системы.

В настоящее время тема проектирования микросервисных платформ с единой аутентификацией и API-шлюзом является одной из приоритетных в области разработки корпоративных и облачных решений. Использование современных архитектурных подходов позволяет повысить устойчивость информационных систем, упростить их масштабирование и обеспечить высокий уровень информационной безопасности.

Целью дипломной работы является разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом. Для достижения поставленной цели необходимо решить следующие задачи:

- изучить современные подходы к построению микросервисных архитектур и цифровых платформ;
- провести анализ предметной области и существующих технических решений;
- определить функциональные и нефункциональные требования к системе;
- разработать архитектуру ядра цифровой экосистемы;
- спроектировать программные компоненты системы (службу аутентификации, API-шлюз, механизмы взаимодействия сервисов);
- реализовать программное решение;
- провести тестирование и оценку эффективности разработанной системы.

Внедрение разработанного решения позволит создать универсальную основу для построения цифровых сервисов различного назначения, повысить управляемость и безопасность инфраструктуры, а также обеспечить возможность дальнейшего масштабирования и развития экосистемы в условиях цифровой трансформации.

1 Теоретическая часть

1.1 Описание предметной области

Предметной областью данного программного обеспечения является ядро цифровой экосистемы, построенной на основе микросервисной архитектуры, а также организация централизованной аутентификации, авторизации и взаимодействия сервисов через API-шлюз.

Цифровая экосистема представляет собой совокупность взаимосвязанных сервисов, объединённых общей архитектурной платформой и едиными механизмами управления доступом. В современных условиях такие экосистемы используются для предоставления широкого спектра цифровых услуг в рамках единой инфраструктуры. Ключевым элементом экосистемы является её ядро, обеспечивающее централизованную обработку запросов, проверку прав доступа пользователей, маршрутизацию трафика и координацию взаимодействия между микросервисами.

Организация работы цифровой экосистемы включает в себя решение задач аутентификации и авторизации пользователей, управления ролями и правами доступа, безопасного обмена данными между сервисами, мониторинга состояния компонентов системы и обеспечения масштабируемости. Важной частью является формирование единой точки входа в систему посредством API-шлюза, что позволяет упростить клиентское взаимодействие и повысить уровень информационной безопасности.

Одной из актуальных проблем в области разработки распределённых систем является сложность обеспечения согласованной работы множества сервисов при сохранении гибкости архитектуры. При отсутствии централизованного ядра возникают трудности с управлением доступом, дублирование функциональности, усложнение сопровождения и масштабирования системы. Кроме того, разрозненные механизмы аутентификации и прямое взаимодействие сервисов повышают риски безопасности и увеличивают затраты на поддержку инфраструктуры.

Разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом позволяет решить указанные проблемы. Такое решение обеспечивает модульность, расширяемость и отказоустойчивость системы, централизованное управление пользователями и безопасную маршрутизацию запросов. Оно также создаёт основу для дальнейшего подключения новых сервисов без существенной переработки существующей архитектуры.

Данное программное обеспечение может быть востребовано в организациях различного масштаба и сферы деятельности – от стартапов до крупных корпоративных структур, развивающих собственные цифровые платформы. Разработанное ядро может быть интегрировано в существующую IT-инфраструктуру, обеспечивая технологическую основу для построения и развития современной цифровой экосистемы.

1.2 Анализ существующей ситуации

Существующая ситуация в области разработки и эксплуатации цифровых сервисов характеризуется высокой степенью фрагментации архитектурных решений. В разных организациях и проектах используются различные подходы к построению распределённых систем, однако можно выделить ряд общих проблем, которые могут быть эффективно решены за счёт разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом:

1. Разрозненность сервисов и отсутствие единой точки входа. Во многих системах сервисы разрабатываются и развёртываются независимо друг от друга без централизованного механизма маршрутизации запросов. Это приводит к усложнению клиентского взаимодействия, необходимости прямого обращения к различным сервисам и повышению рисков безопасности.
2. Отсутствие централизованной аутентификации и управления доступом. Использование отдельных механизмов входа в каждом сервисе

приводит к дублированию функциональности, усложнению сопровождения и повышенной вероятности ошибок в реализации безопасности. Отсутствие единой системы управления ролями и правами доступа затрудняет контроль над пользовательскими действиями.

3. Сложности масштабирования и сопровождения системы. При росте количества сервисов и пользователей нагрузка на инфраструктуру увеличивается. Без продуманной микросервисной архитектуры и API-шлюза возникают трудности с балансировкой нагрузки, обновлением компонентов и обеспечением отказоустойчивости.
4. Недостаточная прозрачность и управляемость инфраструктуры. Отсутствие централизованного логирования, мониторинга и унифицированной обработки ошибок усложняет выявление проблем, анализ инцидентов и контроль состояния системы в целом.
5. Трудности интеграции новых сервисов. Без наличия архитектурного ядра подключение новых модулей требует значительных изменений в существующих компонентах, что снижает гибкость системы и увеличивает затраты на развитие платформы.

В целом, разработка ядра цифровой экосистемы с использованием микросервисной архитектуры, централизованной аутентификации и API-шлюза позволяет устранить перечисленные проблемы. Такое решение обеспечивает единый механизм управления доступом, упрощает масштабирование и сопровождение системы, повышает уровень информационной безопасности и создаёт технологическую основу для устойчивого развития цифровой платформы.

As is, to be

1.3 Постановка задачи

Целью дипломной работы является разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой

аутентификацией и API-шлюзом. Одним из ключевых требований к разрабатываемой системе является обеспечение унифицированного взаимодействия между микросервисами и внешними клиентскими приложениями с использованием стандартизированных форматов обмена данными, таких как JSON, что позволяет упростить интеграцию и повысить совместимость с другими системами и платформами.

Для достижения поставленной цели необходимо последовательно решить ряд задач, каждая из которых реализуется с применением соответствующих методов и подходов.

На первом этапе выполняется изучение современных подходов к построению микросервисных архитектур и цифровых платформ. Для этого проводится анализ научной литературы, технической документации и практик разработки, включая принципы построения распределённых систем, контейнеризации, сервисной декомпозиции и организации взаимодействия между сервисами.

Далее осуществляется анализ предметной области и существующих технических решений. В рамках данного этапа рассматриваются современные платформы и инструменты, реализующие API-шлюзы, системы аутентификации и управления доступом. Анализ позволяет выявить их преимущества, недостатки и особенности реализации, которые будут учтены при проектировании собственной системы.

На следующем этапе производится определение функциональных и нефункциональных требований к разрабатываемой системе. Функциональные требования описывают основные возможности системы, такие как регистрация и аутентификация пользователей, маршрутизация запросов и взаимодействие микросервисов. Нефункциональные требования включают требования к производительности, масштабируемости, безопасности, отказоустойчивости и удобству сопровождения.

После формализации требований разрабатывается архитектура ядра цифровой экосистемы. На данном этапе определяется структура системы,

выделяются основные компоненты, такие как API-шлюз, служба аутентификации, сервис авторизации и микросервисы, а также описываются их взаимодействия. При проектировании используются принципы микросервисной архитектуры, включая слабую связанность компонентов и независимость их развертывания.

Следующим шагом является проектирование программных компонентов системы. Для этого разрабатываются модели данных, интерфейсы взаимодействия сервисов, а также логика работы ключевых модулей, включая механизм аутентификации пользователей, генерацию и проверку токенов, обработку и маршрутизацию API-запросов. Также определяется формат обмена данными (JSON) и структура API.

На этапе реализации производится разработка программного решения с использованием выбранных технологий. Реализуются основные компоненты системы, включая API-шлюз, сервис аутентификации и базовые микросервисы. Особое внимание уделяется обеспечению корректного взаимодействия между компонентами, обработке ошибок и логированию.

Заключительным этапом является тестирование и оценка эффективности разработанной системы. Проводится проверка корректности работы всех компонентов, тестирование сценариев аутентификации и авторизации, а также оценка производительности и устойчивости системы при обработке запросов. По результатам тестирования формулируются выводы о качестве разработанного решения и его соответствии поставленным требованиям.

1.4 Анализ существующих разработок и обоснование актуальности нового проекта

На рынке современных цифровых платформ уже существуют решения, ориентированные на построение распределённых систем и микросервисных экосистем с централизованной аутентификацией и API-шлюзом. Эти решения обеспечивают унифицированное взаимодействие между сервисами,

управление доступом пользователей и маршрутизацию запросов, но различаются по масштабу, архитектуре и набору функций.

Например, компания Amazon Web Services (AWS) предлагает платформу AWS App Mesh и сервисы API Gateway, которые позволяют организовать взаимодействие микросервисов, централизованно управлять доступом и обеспечивать отказоустойчивость распределённых приложений. Эти решения поддерживают маршрутизацию запросов, мониторинг состояния сервисов и масштабирование инфраструктуры.

Другой пример – платформа Kong Inc., предоставляющая API-шлюз с функциями управления доступом, аутентификации, лимитирования запросов и логирования. Решение ориентировано на обеспечение безопасного и управляемого взаимодействия между клиентскими приложениями и микросервисами, а также на упрощение интеграции новых сервисов в существующую архитектуру.

Существуют также комплексные решения для построения корпоративных цифровых экосистем, такие как Red Hat OpenShift, которые предоставляют инфраструктуру для контейнеризации и оркестрации микросервисов, управление ролями и политиками безопасности, а также инструменты мониторинга и масштабирования приложений.

Анализ существующих решений показывает, что современные платформы обеспечивают гибкость архитектуры, безопасность, масштабируемость и централизованное управление доступом. Однако большинство из них ориентированы на крупные корпоративные инфраструктуры или облачные решения, что делает их сложными для внедрения в качестве универсального ядра цифровой экосистемы для разнопрофильных сервисов.

Таким образом, разработка собственного ядра цифровой экосистемы с микросервисной архитектурой, единой аутентификацией и API-шлюзом позволит создать универсальное, безопасное и масштабируемое решение, обеспечивающее простоту интеграции новых сервисов, централизованное

управление пользователями и эффективное взаимодействие между компонентами платформы.

2 Специальная часть

2.1 Выбор технологий и инструментальных средств

2.1.1 Выбор подхода к разработке

Для разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом наиболее подходящим является компонентно-ориентированный и модульный подход с элементами объектно-ориентированного проектирования.

При создании такой системы важно обеспечить чёткое разделение ответственности между отдельными микросервисами и компонентами ядра, такими как служба аутентификации, API-шлюз, сервис маршрутизации, мониторинга и логирования. Каждый компонент должен иметь свои чётко определённые функции и интерфейсы взаимодействия, что позволяет легко масштабировать систему, подключать новые сервисы и обеспечивать отказоустойчивость.

В рамках компонентно-ориентированного подхода каждый микросервис реализуется как независимый модуль с собственной логикой, данными и API. При этом можно использовать объектно-ориентированные принципы для внутренней организации кода: классы и объекты помогают структурировать функциональность, управлять состоянием сервиса и реализовывать безопасное взаимодействие с другими модулями. Такой подход повышает масштабируемость, гибкость и удобство сопровождения системы, а также упрощает тестирование и отладку отдельных компонентов.

Напротив, использование чисто структурного подхода в разработке распределённых систем с микросервисами может привести к сложностям: повторение кода, тесная связанность компонентов и затруднённое масштабирование, что критично для больших экосистем с высокой нагрузкой.

Таким образом, комбинированный подход – компонентно-ориентированная архитектура с применением принципов ООП внутри микросервисов, является оптимальным для разработки ядра цифровой экосистемы. Он обеспечивает необходимую гибкость, отказоустойчивость, масштабируемость и упрощает интеграцию новых сервисов и технологий в платформу.

2.1.2 Выбор среды разработки и языка программирования

Для разработки ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом необходимо обоснованно выбрать языки программирования, фреймворки и инструменты разработки. Выбор технологий должен основываться на сравнительном анализе существующих решений с учётом требований к масштабируемости, производительности, безопасности и удобству сопровождения.

На этапе анализа были рассмотрены несколько популярных языков программирования для серверной разработки и построения микросервисных систем, включая Python, Java (Spring Boot), Node.js и C# (.NET).

Язык Java и платформа Spring Boot широко применяются при разработке корпоративных систем благодаря высокой производительности, развитой экосистеме и встроенным механизмам безопасности. Однако они требуют большего объёма кода и обладают более высоким порогом вхождения, что может усложнить и замедлить разработку.

Node.js обеспечивает высокую производительность при обработке большого количества асинхронных запросов и активно используется при разработке API. Тем не менее, динамическая типизация JavaScript может приводить к ошибкам на этапе выполнения и усложнять поддержку крупных проектов.

Платформа .NET (C#) предоставляет мощные инструменты разработки, высокую производительность и хорошую интеграцию с корпоративными

решениями. Однако она в большей степени ориентирована на экосистему Microsoft и может требовать дополнительных ресурсов для развертывания.

Язык Python, в свою очередь, отличается простотой синтаксиса, высокой скоростью разработки и наличием большого количества библиотек для работы с веб-сервисами, безопасностью и взаимодействием компонентов распределённых систем. Несмотря на несколько меньшую производительность по сравнению с Java или C#, Python является оптимальным выбором для разработки прототипов и систем средней нагрузки, особенно при использовании современных асинхронных фреймворков.

В результате проведённого анализа в качестве основного языка программирования выбран Python.

Для разработки REST API также были рассмотрены фреймворки FastAPI, Flask и Django. Фреймворк Django является мощным решением с большим количеством встроенных возможностей, однако он избыточен для микросервисной архитектуры и менее гибок в контексте лёгких сервисов.

Flask представляет собой минималистичный фреймворк, позволяющий гибко настраивать архитектуру приложения, однако требует подключения большого количества сторонних компонентов.

FastAPI сочетает простоту разработки, высокую производительность за счёт асинхронной обработки запросов и встроенную поддержку валидации данных и генерации документации. В связи с этим он является наиболее предпочтительным для реализации микросервисов.

Для обеспечения изоляции компонентов системы и упрощения процесса развертывания были рассмотрены подходы виртуализации и контейнеризации. Использование Docker позволяет упаковывать каждый микросервис в отдельный контейнер, обеспечивая независимость компонентов и удобство масштабирования. Для управления несколькими контейнерами целесообразно использовать Docker Compose, а при необходимости масштабирования и управления кластером — Kubernetes.

В качестве системы управления базами данных были рассмотрены PostgreSQL и MySQL. Обе СУБД являются надёжными и широко используемыми решениями. PostgreSQL обладает более широкими возможностями расширения, поддержкой сложных типов данных и лучшей работой с транзакциями, что делает её предпочтительной для данной системы.

Для повышения производительности и уменьшения задержек при обработке запросов предлагается использовать Redis в качестве системы кэширования, в частности для хранения токенов и часто запрашиваемых данных.

В качестве среды разработки выбрана Visual Studio Code, которая обеспечивает поддержку Python, интеграцию с Docker, системой контроля версий Git, а также предоставляет удобные инструменты для отладки, тестирования и сопровождения кода.

Основные преимущества выбранных технологий заключаются в следующем:

1. Гибкость и масштабируемость. Использование микросервисной архитектуры и контейнеризации позволяет легко расширять систему и добавлять новые компоненты без изменения существующих.
2. Надёжность и отказоустойчивость. Изоляция сервисов и возможность их независимого развертывания повышают устойчивость системы к сбоям.
3. Высокая скорость разработки. Python и FastAPI позволяют быстро реализовывать функциональность при сохранении читаемости и структурированности кода.
4. Удобство интеграции. Использование REST API и формата JSON обеспечивает простое взаимодействие между сервисами и внешними приложениями.
5. Развитая экосистема. Выбранные технологии имеют широкую поддержку сообщества и большое количество готовых решений.

2.2 Разработка спецификаций

2.2.1 Разработка диаграмм вариантов использования

Диаграмма вариантов использования отражает функции ядра платформы: аутентификацию, авторизацию, маршрутизацию запросов, управление сервисами и администрирование. Она представлена на рисунке 1.

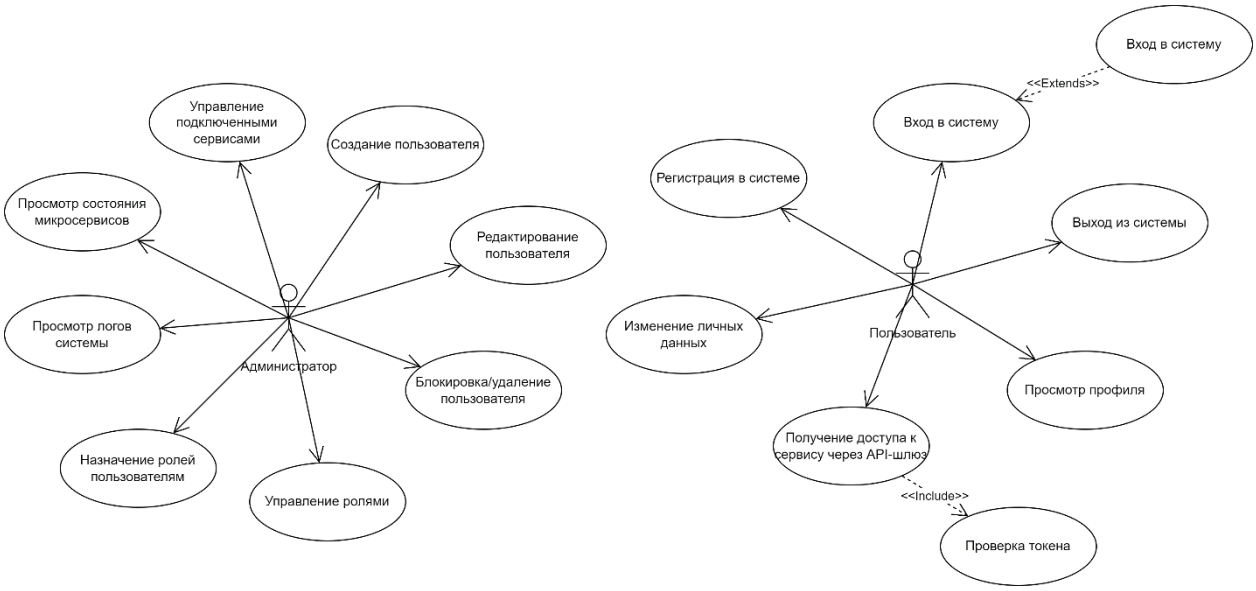

Рисунок 1 – Диаграмма вариантов использования

2.2.2 Разработка диаграмм последовательностей системы

Диаграмма последовательности показывает какой путь проходит запрос от пользователя через все ключевые компоненты ядра. На рисунке 2 представлена диаграмма последовательности аутентификации пользователя.

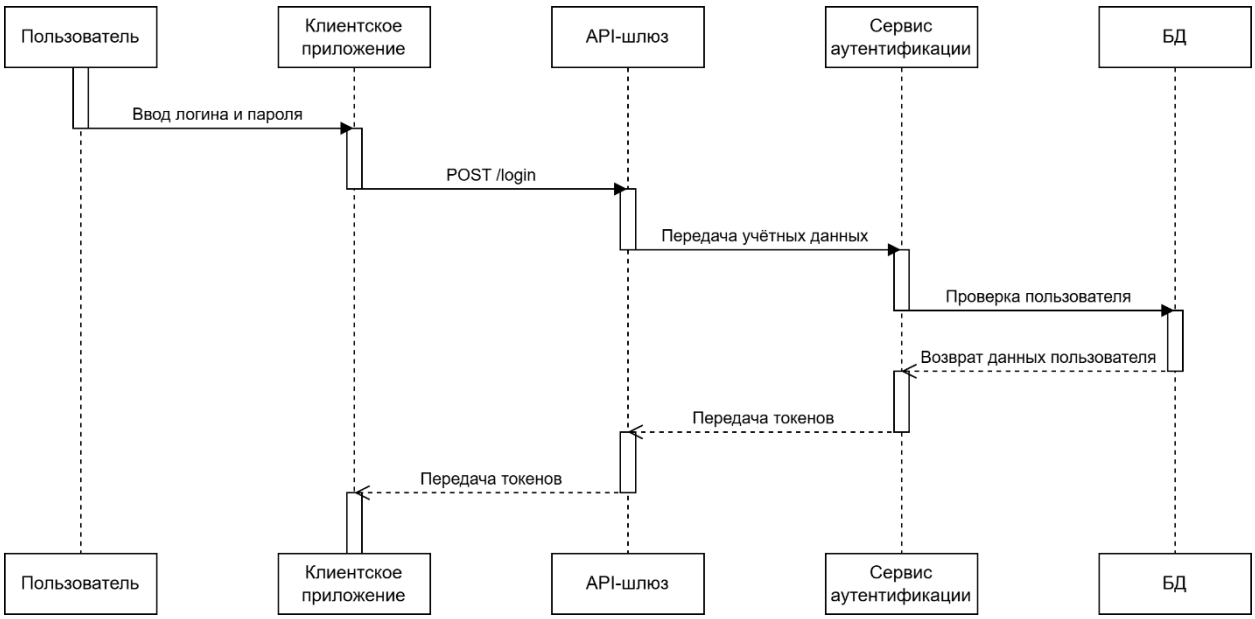

Рисунок 2 – Диаграмма последовательности аутентификации

На рисунке 3 представлена диаграмма последовательности доступа к микросервису.

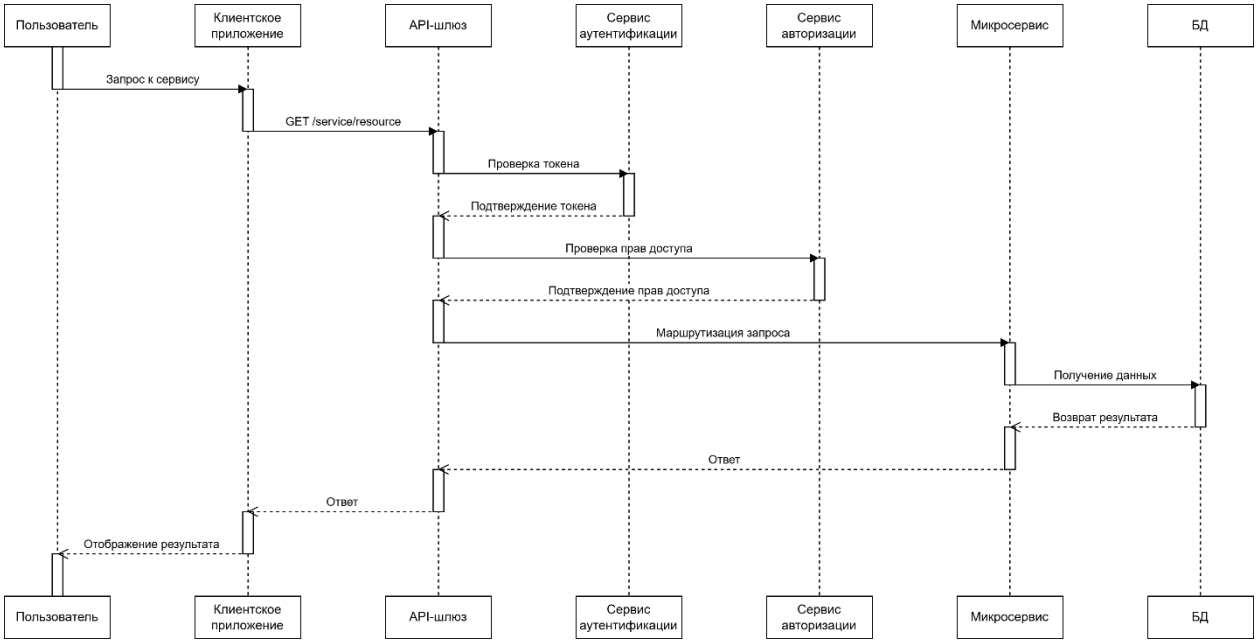

Рисунок 3 – Диаграмма последовательности доступа к микросервису

2.2.3 Разработка диаграммы пакетов

Диаграмма пакетов показывает архитектурную декомпозицию, а также логическую структуру системы и зависимости между её крупными модулями. Диаграмма пакетов представлена на рисунке 4.

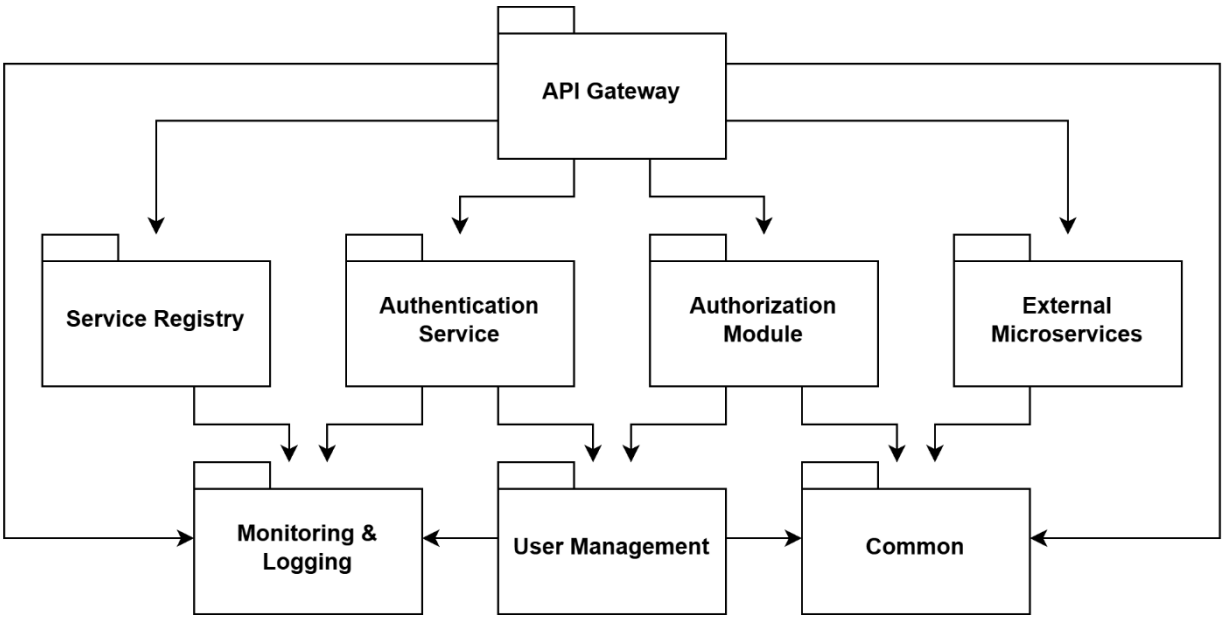

Рисунок 4 – Диаграмма пакетов

Пакеты на диаграмме:

1. «API Gateway» – приём всех входящих запросов, проверка токенов, маршрутизация запросов, ограничение частоты запросов, централизованная обработка ошибок.
2. «Authentication Service» – регистрация пользователей, аутентификация, генерация Access/Refresh токенов, обновление токенов, проверка валидности токенов.
3. «Authorization Module» – управление ролями, назначение прав, проверка доступа к ресурсам, реализация RBAC.
4. «User Management» – CRUD пользователей, хранение профилей, блокировка пользователей, управление статусами.
5. «Service Registry» – регистрация микросервисов, хранение информации о сервисах, предоставление данных для маршрутизации.
6. «Monitoring & Logging» – логирование запросов, сбор метрик, Health-check сервисов, отслеживание ошибок.
7. «Common» – общие модели данных, DTO, утилиты, константы, обработчики исключений.
8. «External Microservices» – бизнес-сервисы, подключённые к ядру, не входят в ядро напрямую, взаимодействуют только через API Gateway.

2.2.4 Разработка диаграммы классов

Диаграмма классов отражает внутреннюю структуру ключевых компонентов ядра. Она представлена на рисунке 5.

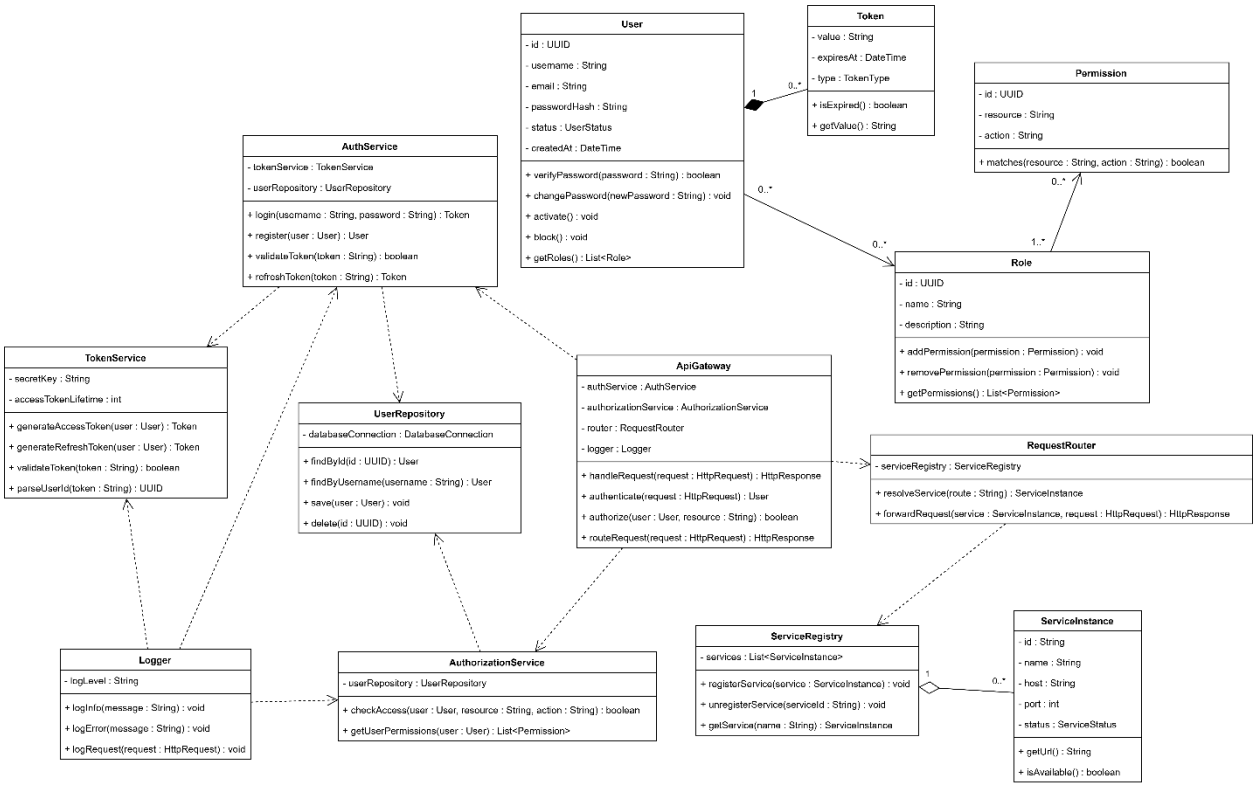

Рисунок 5 – Диаграмма классов

2.2.5 Разработка диаграммы состояний

Диаграмма состояний описывает изменение состояния запроса при его обработке системой. Она показывает, какие этапы проходит запрос в ядре цифровой экосистемы и какие условия вызывают переход между состояниями. Диаграмма состояний представлена на рисунке 6.

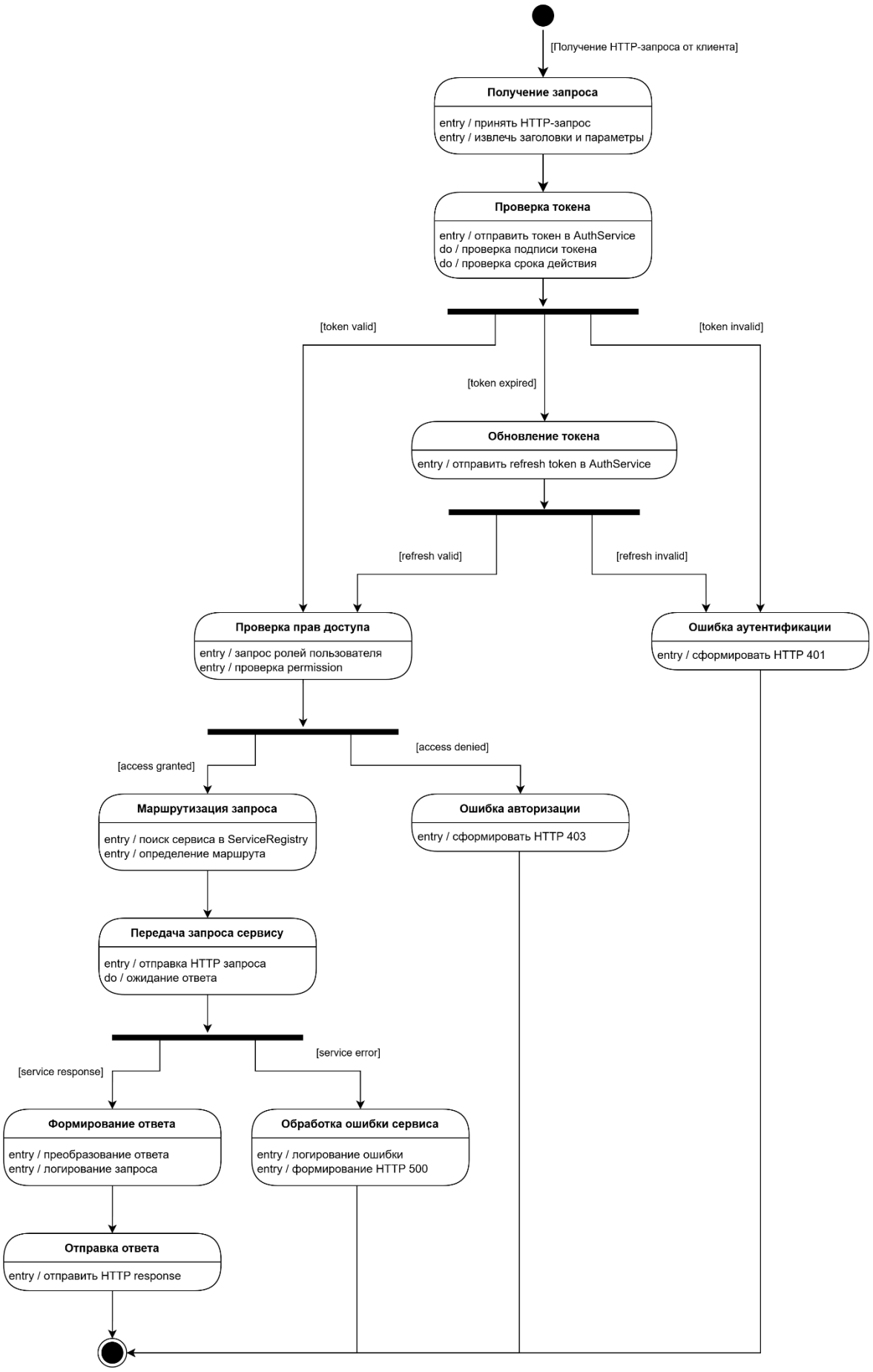

Рисунок 6 – Диаграмма состояний

2.2.6 Разработка диаграммы деятельности

Диаграмма деятельности описывает процесс обработки пользовательского запроса в системе. Она показывает последовательность действий, принимаемые решения, возможные ветвления и взаимодействие компонентов системы. Диаграмма деятельности представлена на рисунке 7.

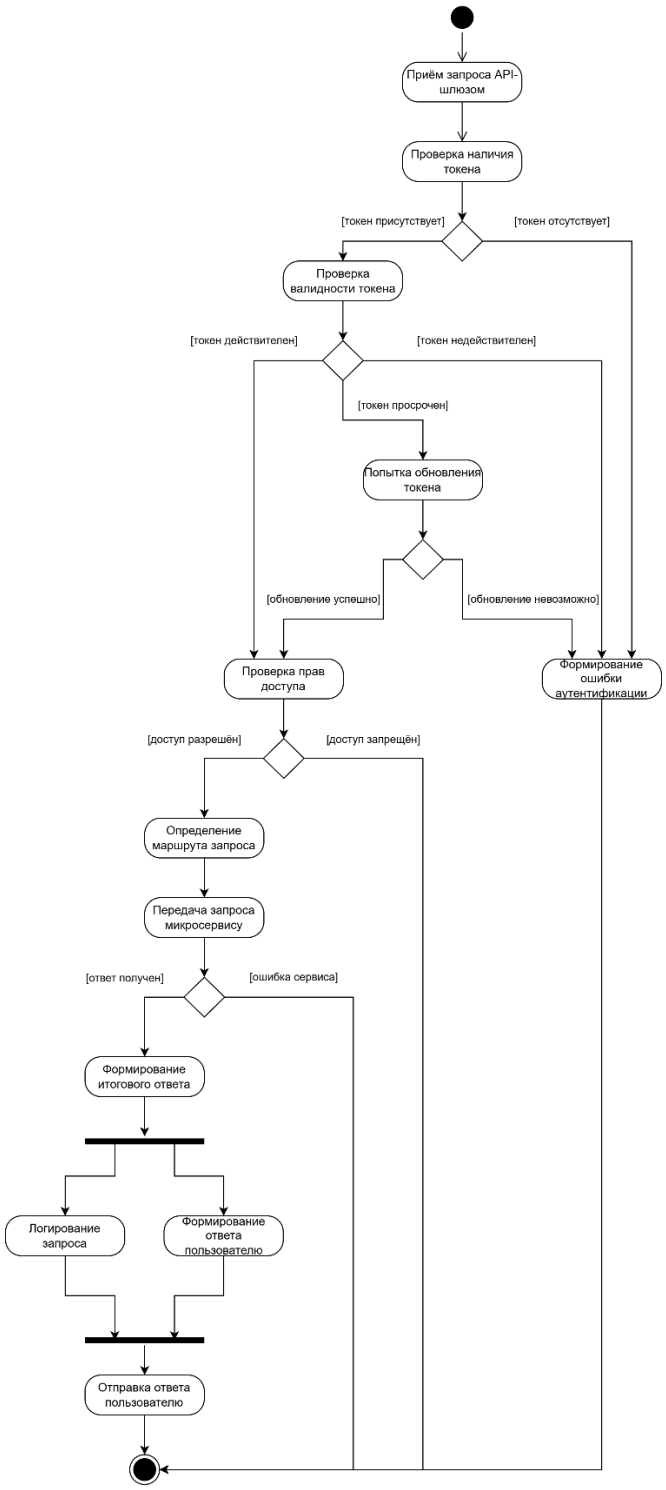

Рисунок 7 – Диаграмма деятельности

2.2.8 Разработка инфологической модели базы данных

Инфологическая модель – это абстрактное представление о данных, которые будут храниться в базе данных. Она описывает данные и их связи между собой, но не учитывает способ их хранения. Эта модель используется для определения структуры базы данных и ее функциональности. Инфологическая модель базы данных представлена на рисунке 8.

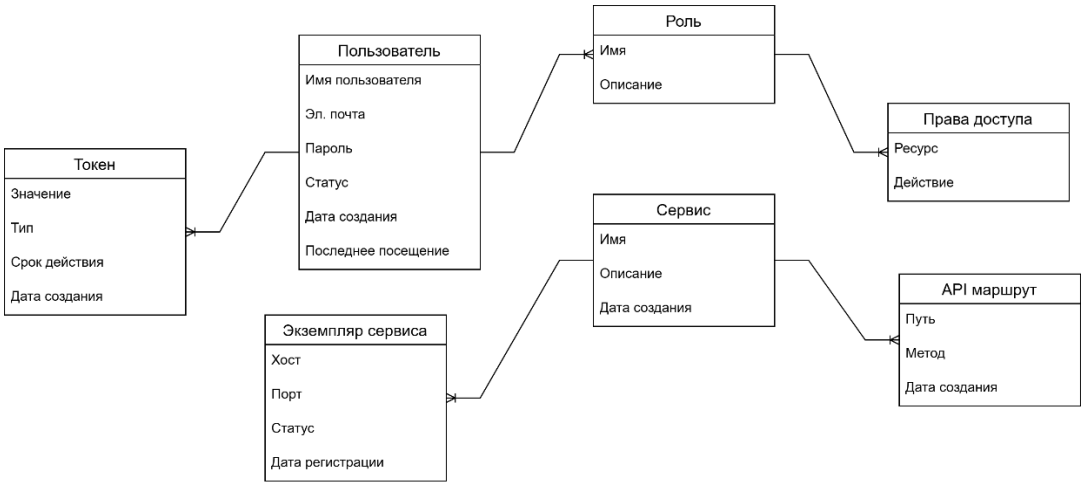

Рисунок 8 – Инфологическая модель базы данных

2.2.9 Разработка физической модели базы данных

Физическая модель базы данных предназначена для хранения данных о пользователях системы, их ролях и правах доступа, токенах аутентификации, а также информации о зарегистрированных микросервисах. База данных обеспечивает централизованное управление безопасностью и инфраструктурой системы. Физическая модель базы данных представлена на рисунке 9.

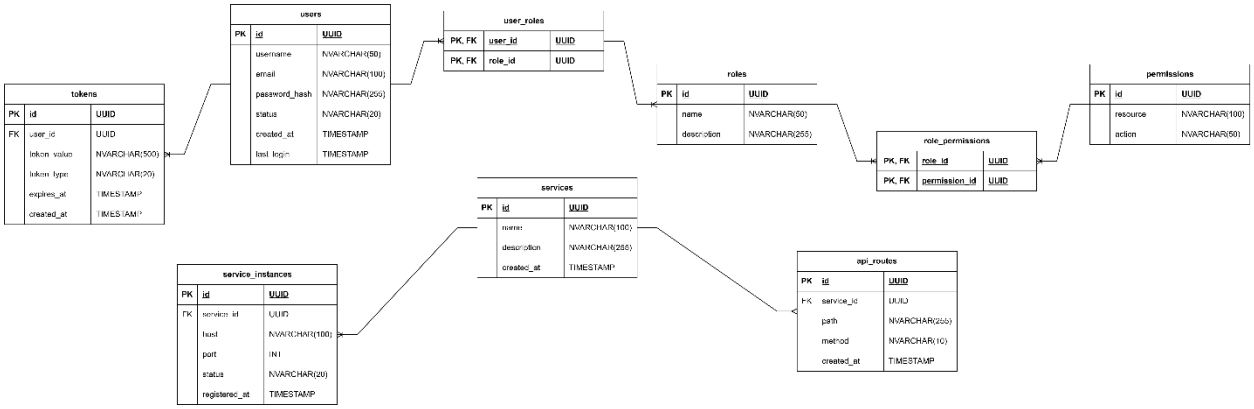

Рисунок 9 – Физическая модель базы данных

2.2.10 Разработка диаграммы размещения

Диаграмма размещения описывает физическое распределение компонентов системы по узлам инфраструктуры и показывает, как программные модули развёрнуты и взаимодействуют между собой. Диаграмма размещения представлена на рисунке 10.

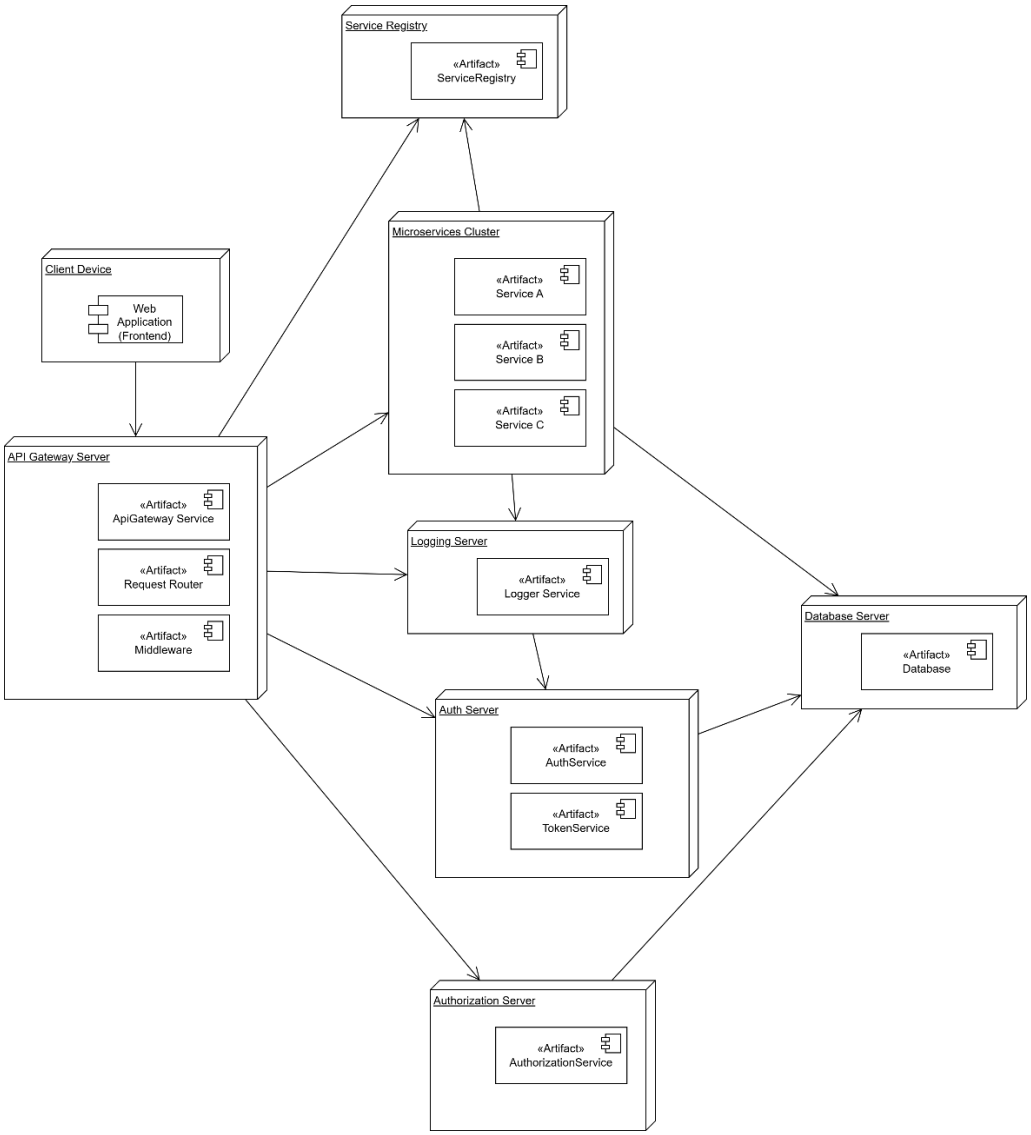

Рисунок 10 – Диаграмма размещения

2.3 Проектирование программного модуля

2.3.1 Разработка алгоритмов реализации основных функций программного обеспечения

2.3.2 Проектирование пользовательского интерфейса

2.4 Реализация программного обеспечения на выбранном языке программирования и в выбранной среде разработки

2.5 Выбор стратегии тестирования, разработка тестов, тестирование и отладка программного обеспечения

2.6 Разработка эксплуатационной документации

2.6.1 Разработка руководства системного программиста

2.6.2 Разработка руководства пользователя

3 Экономическая часть

Для проведения расчетов трудоемкости разработки программного продукта и составления сметы затрат необходимо определить исходные данные.

1. Показатели объема и сложности программы:

Базовым показателем для расчета трудоемкости является предполагаемое число операторов в программе - 2000 операторов (строк кода).

Для определения условного числа операторов используются коэффициенты сложности и новизны, а также коэффициент коррекции. Коэффициент коррекции учитывает новизну разрабатываемого продукта и для новой программы принимается равным 0,1.

Разрабатываемый программный продукт относится к 1-й группе сложности, так как включает алгоритмы оптимизации и элементы моделирования систем. Степень новизны соответствует категории В - разработка программы с использованием типовых решений. Для данной группы сложности и степени новизны коэффициент сложности (Кс) принимается равным 1,15.

Коэффициент увеличения затрат труда (Кув.з.), необходимый для расчета трудоемкости исследования алгоритма, составляет 1,20.

2. Показатели для расчета трудоемкости:

Для расчета трудоемкости отдельных этапов используются следующие показатели:

- Число операторов, приходящихся на один человеко-час (Ч1) - 75 операторов/чел.час;
- Коэффициент квалификации разработчика (Ккв.р.) - 0,8.

3. Показатели для расчета заработной платы:

Месячная заработная плата программиста составляет 60 000 рублей. Норматив отчислений на социальные нужды установлен в размере 30%. Месячный фонд рабочего времени при 40-часовой рабочей неделе

принимается равным 168 часам. Норматив дополнительной заработной платы составляет 16% от основной заработной платы.

4. Показатели для расчета затрат на машинное время:

Срок службы компьютера, на котором ведется разработка, составляет 5 лет. Балансовая стоимость компьютера принимается в соответствии с заданием и равна 80 000 рублей. Норма амортизации при линейном методе начисления составляет 20% в год.

Действительный годовой фонд времени работы компьютера при продолжительности рабочего дня 8 часов составляет 2 112 часов в год.

Для расчета затрат на текущий ремонт и прочие расходы применяются коэффициенты, установленные методическими указаниями:

- Коэффициент для расчета издержек на вспомогательные материалы - 0,01 от балансовой стоимости;
- Коэффициент для расчета затрат на текущий ремонт - 0,05 от балансовой стоимости;
- Коэффициент для расчета прочих и накладных расходов - 0,06 от балансовой стоимости.

5. Показатели для расчета затрат на электроэнергию:

Тариф на электроэнергию принимается в соответствии с заданием и составляет 7,28 рублей за киловатт-час. Мощность, потребляемая компьютером, используемым для разработки, принимается равной 0,45 кВт.

6. Показатели для расчета экономической эффективности:

Для оценки экономической эффективности необходимы дополнительные исходные данные, характеризующие базовый вариант решения задачи (без использования разрабатываемого программного продукта).

Средняя часовая заработная плата специалиста, выполняющего каталогизацию фотографий вручную, принимается равной 600 рублей в час. Данное значение является условным и рекомендуется методическими указаниями для учебных расчетов.

Доля заработной платы в общей смете затрат организации устанавливается в пределах 0,45-0,55. Для расчетов принимается значение 0,45.

Для решения задачи без использования программного продукта требуется определенная доля действующего фонда рабочего времени. На основе анализа трудоемкости работы без внедрения микросервисной экосистемы принимаем Нтр = 45%.

Срок службы программного продукта принимается равным 5 годам.

Для удобства дальнейших расчетов все исходные показатели сведены в таблицу 1.

Таблица 1 – Исходные данные

Наименование показателя	Значение
Предполагаемое число операторов, Чпр.	2 000 шт.
Коэффициент сложности задачи, Кс.	1,15 отн.ед.
Коэффициент коррекции программы, Ккор.	0,1 отн.ед.
Коэффициент увеличения затрат труда, Кув.з.	1,20 отн.ед.
Число операторов на один чел.час., Ч1.	75 шт./чел.час
Коэффициент квалификации разработчика, Ккв.р.	0,8 отн.ед.
Месячная заработная плата программиста, Ппр.	60 000 руб.
Месячный фонд рабочего времени, Фр.в.	168 часов
Норматив дополнительной зарплаты, Нд.з.п.	16%
Норматив отчислений на соцнужды, Но.с.н.	30%
Балансовая стоимость компьютера, Сбал.	80 000 руб.
Норма амортизации, На.	20%
Годовой фонд времени компьютера, Тп.к.	2 112 часов
Стоимость одного кВт.-часа электроэнергии, С1.	7,28 руб./кВт*ч
Потребляемая мощность, Р.	0,45 кВт
Средняя часовая зарплата, Сч.	600 руб./час
Доля зарплаты в общей смете затрат, Дз.п.	0,45 отн.ед.

Доля фонда времени на решение задачи, Нтр.	45%
Срок службы программного продукта, Тс.	5 лет

3.1 Расчет трудоемкости разработки программного продукта

Трудоемкость разработки программного продукта определяется суммой затрат на каждом этапе работы: описание задачи, исследование алгоритма, разработка блок-схемы, программирование, отладка программы и подготовка документации.

1. Расчет условного числа операторов:

Первоначально рассчитывается условное число операторов в программе. Этот показатель учитывает сложность будущего продукта и необходимость внесения правок в процессе разработки:

$$Чоп.усл. = Чпр. * Кс. * (1 + Ккор.) \quad (2.1)$$

где: Чпр. – предполагаемое число операторов (строк) в программе, шт.;

Кс. – коэффициент сложности задачи;

Ккор. – коэффициент коррекции программы, отн.ед.

Подставляем значения:

Чпр. = 2 000 операторов;

Кс. = 1,15 отн.ед;

Ккор. = 0,1 отн.ед.

Чоп.усл. = 2 000 * 1,15 * (1 + 0,1) = 2 530 операторов

2. Затраты труда на описание задачи:

Для определения затрат на описание задачи используется метод усреднения трех оценок: оптимистической, наиболее вероятной и пессимистической. Так как условное число операторов попадает в диапазон 1 901–2 400, используем соответствующие значения из таблицы:

Тмин. = 40 н.час;

Тн.в. = 70 н.час;

Тмакс. = 100 н.час.

Расчет выполняется по формуле:

$$To = (Tмин. + 4 * Tн.в. + Tмакс.) / 6 \qquad (2.2)$$

где: Тмин. – трудоёмкость операции при благоприятных условиях (оптимистическая оценка), н.час.;

Тн.в. – трудоёмкость операции при нормальных условиях (наиболее вероятная оценка), н.час.;

Тмакс. – трудоёмкость операции в наиболее неблагоприятных условиях (пессимистическая оценка), н.час.;

Подставляем значения:

To = (40 + 4 * 70 + 100) / 6 = 70 н.час.

3. Затраты труда на исследование алгоритма решения задачи:

Затраты на исследование алгоритма зависят от объема программы, сложности задачи и квалификации разработчика:

$$Tи = (Чоп.усл. * Кув.з.) / (Ч1 * Ккв.р.) \qquad (2.3)$$

где: Чоп.усл. – условное число операторов в программе, шт.;

Кув.з. – коэффициент увеличения затрат в зависимости от сложности программы;

Ч1 – число операторов, приходящихся на один чел.час. (75 . . . 85);

Ккв.р. - коэффициент квалификации разработчика.

Подставляем значения:

Чоп.усл. = 2 530 оператор;

Кув.з. = 1,20 отн.ед;

Ч1 = 75 шт/чел.час;

Ккв.р. = 0,8 отн.ед.

Ти = (2 530 * 1,20) / (75 * 0,8) = 51 чел.час.

4. Затраты труда на разработку блок-схемы алгоритма:

Затраты на построение блок-схемы алгоритма рассчитываются исходя из условного числа операторов и квалификации разработчика:

$$Tа = Чоп.усл. / (20 * Ккв.р.) \qquad (2.4)$$

Подставляем значения:

Та = 2 530 / (20 * 0,8) = 158 чел.час.

5. Затраты труда на отладку программы:

Отладка программы является наиболее трудоемким этапом разработки. Затраты на отладку рассчитываются по формуле:

$$Тотл. = Чоп.усл. / (4…5 * Ккв.р.) \qquad (2.5)$$

Подставляем значения:

Тотл. = 2 530 / (4,5 * 0,8) = 703 чел.час.

При комплексной отладке, включающей проверку всех модулей в их взаимодействии, затраты увеличиваются в 1,2 раза:

Тотл.окон. = Тотл. * 1,2 = 703 * 1,2 = 843 чел.час.

6. Затраты труда на подготовку документации:

Подготовка документации включает написание текстовых материалов и их последующее оформление. Затраты на подготовку рукописных материалов:

$$Тд.р. = Чоп.усл. / ((15...20) * Ккв.р.) \qquad (2.6)$$

Подставляем значения:

Тд.р. = 2 530 / (17 * 0,8) = 186 чел.час.

Затраты на редактирование, печать и оформление документации составляют 75% от затрат на подготовку рукописи:

Тд.о. = 0,75 * Тд.р. = 0,75 * 186 = 140 чел.час.

Общие затраты на подготовку документации:

Тд = Тд.р. + Тд.о. = 186 + 140 = 326 чел.час.

7. Затраты труда на программирование:

Затраты на написание исходного кода программы составляют 30% от суммарных затрат на все остальные этапы разработки:

$$Тп = (То + Ти + Та + Тотл.окон. + Тд) * (25. . .35) / 100 \qquad (2.7)$$

Подставляем значения:

Тп = (70 + 51 + 158 + 843 + 326) * 30 / 100 = 434 чел.час.

8. Общая трудоемкость разработки:

Общие затраты труда на разработку программного продукта определяются суммированием затрат по всем этапам:

$$T = To + Ти + Та + Тотл.окон. + Тд + Тп \quad (2.8)$$

Подставляем значения:

T = 70 + 51 + 158 + 843 + 326 + 434 = 1 882 чел.час.

Полученные результаты по затратам труда на разработку программного продукта сведены в таблицу 2. Удельный вес каждого этапа показывает, какую долю в общей трудоемкости занимает тот или иной вид работ.

Таблица 2 – Доли работ в общей трудоемкости

Виды затрат труда	Индекс	Трудоемкость, чел.час	Удельный вес затрат, %
Описание задачи	То	70	3,7
Исследование алгоритма	Ти	51	2,7
Разработка блок-схемы	Та	158	8,4
Отладка программы	Тотл.окон.	843	44,8
Подготовка документации	Тд	326	17,3
Программирование	Тп	434	23,1
Итог	Т	1 882	100

Из таблицы видно, что наиболее трудоемкими этапами являются отладка программы (44,8%) и программирование (23,1%). Высокая доля отладки объясняется сложностью разрабатываемого продукта и необходимостью тщательной проверки работы алгоритмов передачи данных между сервисами, а также их взаимодействия с остальными модулями платформы.

3.2. Составление сметы затрат на разработку

Смета затрат на разработку программного продукта включает в себя следующие статьи: затраты на оплату труда программиста, отчисления на социальные нужды, затраты на оплату машинного времени, затраты на электроэнергию и прочие затраты.

1. Расчет затрат на оплату труда разработчика:

Для определения затрат на оплату труда необходимо рассчитать часовую ставку программиста. Месячная заработная плата программиста составляет 60 000 рублей. При месячном фонде рабочего времени 168 часовая ставка рассчитывается по формуле:

$$Сч.пр. = Ппр. / Фр.в. \quad (2.9)$$

где: Ппр. – месячная заработная плата программиста, руб.

Фр.в. – месячный фонд рабочего времени, час. (168 час.)

Подставляем значения:

Сч.пр. = 60 000 / 168 = 357 руб./час

Основная заработная плата разработчика определяется произведением часовой ставки на общую трудоемкость разработки:

$$ЗПосн. = Сч.пр. * Т \quad (2.10)$$

где: Сч.пр. – часовая оплата труда программиста, руб./ час.

Т – трудоемкость разработки программного продукта, чел-час.;

Подставляем значения:

ЗПосн. = 357 * 1 882 = 672 105 руб.

Дополнительная заработная плата включает выплаты за непроработанное время (отпуска, выполнение государственных обязанностей и т.п.) и рассчитывается по нормативу 16% от основной заработной платы:

$$ЗПдоп. = ЗПосн. * Нд.з.п. / 100 \quad (2.11)$$

где: Нд.з.п. – норматив дополнительной заработной платы, %. (15. . .19 %).

Подставляем значения:

ЗПдоп. = 672 105 * 16 / 100 = 107 537 руб.

Отчисления на социальные нужды производятся в соответствии с действующим законодательством по нормативу 30% от суммы основной и дополнительной заработной платы:

Ос.н. = (ЗПосн. + ЗПдоп.) * Но.с.н. / 100 (2.12)

где: Но.с.н. – норматив отчислений на социальные нужды, % (30 %)

Подставляем значения:

Ос.н.= (672 105 + 107 537) * 30 / 100 = 233 892 руб.

Общая заработная плата разработчика (фонд оплаты труда с отчислениями) составит:

ЗПобщ. = ЗПосн. + ЗПдоп. + Ос.н. (2.13)

где: ЗПосн. – основная заработная плата разработчика, руб.,

ЗПдоп. – дополнительная заработная плата разработчика, ру.,

Ос.н. – отчисления на социальные нужды, руб.

Подставляем значения:

ЗПобщ. = 672 105 + 107 537+ 233 892 = 1 013 534 руб.

2. Расчет затрат на оплату машинного времени

Для расчета затрат на машинное время необходимо определить стоимость одного машино-часа работы компьютера. В этой стоимости учитываются амортизация оборудования, затраты на вспомогательные материалы, текущий ремонт и прочие расходы.

Годовые амортизационные отчисления рассчитываются исходя из балансовой стоимости компьютера 60 000 рублей и нормы амортизации 20%:

За = Сбал. * На / 100 (2.14)

где: Сбал. – балансовая стоимость компьютера, руб.;

На – норма амортизации в процентах.

Подставляем значения:

За = 80 000 * 20 / 100 = 16 000 руб.

Годовые издержки на вспомогательные материалы принимаются в размере 1% от балансовой стоимости компьютера:

Зв.м. = Сбал. * 0,01 (2.15)

Подставляем значения:

Зв.м.= 80 000 * 0,01 = 800 руб.

Затраты на текущий ремонт компьютера принимаются в размере 5% от балансовой стоимости:

Зт.р. = Сбал. *0,05 (2.16)

Подставляем значения:

Зт.р. = 80 000 * 0,05 = 4 000 руб.

Прочие и накладные расходы принимаются в размере 6% от балансовой стоимости:

Зп.р. = Сбал. * 0,06 (2.17)

Подставляем значения:

Зп.р. = 80 000 * 0,06 = 4 800 руб.

Действительный годовой фонд времени работы компьютера составляет 2 112 часов. Стоимость одного машино-часа:

См. = (За + Зв.м. + Зт.р. + Зп.р.) / Тп.к. (2.18)

где: За – затраты на амортизацию, руб. в год;

Зв.м – годовые издержки на вспомогательные материалы, руб.;

Зт.р – затраты на текущий ремонт компьютера, руб. в год;

Зп.р – годовые издержки на прочие и накладные расходы, руб.;

Тп.к – действительный годовой фонд времени ЭВМ, час.

Подставляем значения:

См. = (16 000 + 800 + 4000 + 4800) / 2 112 = 12,12 руб./час

3. Затраты на оплату машинного времени:

Зм.в. = См. * (Тп + Тотл.окон. + Тд) (2.19)

где: См. – стоимость машино-часа, руб.

Подставляем значения:

Зм.в. = 12,12 * (434 + 843 + 326) = 19 432 руб.

4. Расчет затрат на электроэнергию:

Затраты на электроэнергию, потребляемую компьютером в процессе разработки, рассчитываются исходя из тарифа на электроэнергию, потребляемой мощности компьютера и времени работы на компьютере:

Сэл. = С1 * Р * (Тп + Тотл.окон. + Тд) (2.20)

где: С1 – стоимость одного кВт.-часа электроэнергии, руб.;

Р – мощность, потребляемая компьютером, кВт.;

Тп – затраты на программирование, руб.;

Тотл – затраты на отладку, руб.;

Тд – затраты на подготовку документации, руб.

Подставляем значения:

Сэл. = 7,28 * 0,45 * (434 + 843 + 326) = 5 252 руб.

5. Расчет прочих затрат:

Прочие затраты включают расходы на канцелярские товары, расходные материалы, услуги связи и другие мелкие расходы, не учтенные в предыдущих статьях. Они принимаются в размере 6% от суммы основных затрат (заработная плата, машинное время, электроэнергия):

Зосн. = ЗПобщ. + Зм.в. + Сэл. (2.21)

Подставляем значения:

Зосн. = 1 013 534 + 19 432 + 5 252 = 1 038 218 руб.

Прочие затраты:

Зпр. = Зосн. * (0,05. . .0,09) (2.22)

Подставляем значения:

Зпр. = 1 038 218 * 0,06 = 62 293 руб.

6. Общая сметная стоимость разработки:

Общие затраты на разработку программного продукта рассчитываются по формуле:

Зобщ. = ЗПобщ. + Зм.в. + Сэл. + Зпр. (2.23)

Подставляем значения:

Зобщ. = 1 013 534 + 19 432 + 5 252 + 62 293 = 1 100 511 руб.

Полученные результаты сведены в таблицу 3. Удельный вес каждой статьи затрат рассчитан как отношение суммы по данной статье к общей сумме затрат, выраженное в процентах.

Таблица 3 – Доли статей затрат от их общей суммы

Статьи затрат	Индекс	Сумма, руб.	Удельный вес, %
Зароботная плата общая	ЗПобщ.	1 013 534	92,1
Затраты на оплату машинного времени	Зм.в.	19 432	1,8
Затраты на электроэнергию	Сэл.	5 252	0,5
Прочие затраты	Зпр.	62 293	5,7
Итого	Зобщ.	1 100 511	100

3.3. Анализ экономической эффективности при внедрении программного обеспечения

Для определения экономической целесообразности разработки программного продукта необходимо сравнить затраты при решении задачи существующим способом и затраты при использовании разработанной платформы. Экономический эффект представляет собой разницу между этими величинами.

1. Расчет затрат по базовому варианту:

В качестве базового варианта рассматривается выполнение задач без внедрения ядра цифровой экосистемы.

Трудоемкость решаемой задачи в течение года рассчитывается исходя из того, что без внедрения цифровой экосистемы для выполнения задач требуется определенная доля действующего фонда рабочего времени:

Тр. = Тп.к. * Нтр. / 100 (2.24)

Подставляем значения:

Тп.к. = 2 112 чел.час;

Нтр. = 45.

Тр. = 2 112 * 45 / 100 = 950 час/год

Затраты по базовому варианту рассчитываются по формуле:

Зб. = Сч. * Тр. * (1 / Дз.п.) (2.25)

где: Сч. – средняя часовая заработная плата;

Тр. – трудоёмкость решаемой задачи;

Дз.п. – доля заработной платы в общей смете затрат организации.

Подставляем значения:

Сч. = 600 руб./час;

Дз.п. = 0,45 отн.ед.

Зб. = 600 * 950 * (1 / 0,45) = 1 267 200 руб./год

2. Расчет затрат при использовании программного продукта:

Затраты при использовании разработанной платформы рассчитываются с учетом того, что затраты на разработку распределяются на весь срок службы программного продукта, а текущие затраты включают стоимость машинного времени на работу с программой:

Зп.п. = (Тг. * См. + Зобщ.) / Тс (2.26)

где: Тг – время, отводимое на работу с программой, час;

См. - стоимость одного машинного часа, руб.;

Тс - срок службы программного продукта, лет;

Подставляем значения:

Тг. = 2 112 часов;

См. = 12,12 руб./час;

Зобщ. = 1 100 511 руб;

Тс. = 5 лет.

Зп.п. = (2 112 * 12,12 + 1 100 511) / 5 = 225 222 руб./год

3. Расчет экономического эффекта:

Экономический эффект от использования программного продукта рассчитывается как разница между затратами по базовому варианту и затратами при использовании программы:

Э = Зб. - Зп.п. (2.27)

где: Зб. – затраты по базовому варианту, руб./год;

Зп.п. – затраты при использовании программного обеспечения, руб./год.

Подставляем значения:

Э = 1 267 200 - 225 222 = 1 041 978 руб./год

4. Расчет срока окупаемости:

Общие расходы, связанные с созданием и эксплуатацией программного продукта, включают балансовую стоимость компьютера и затраты на разработку:

Робщ. = Сбал. + Зобщ. (2.28)

где: Сбал. – балансовая стоимость компьютера, руб.

Подставляем значения:

Робщ. = 80 000 + 1 100 511 = 1 180 511 руб.

Срок окупаемости рассчитывается по формуле:

Ток. = Робщ. / Э (2.29)

где: Робщ. – сумма общих расходов, руб.,

Подставляем значения:

Ток. = 1 180 511 / 1 041 978 = 1,1 год

Ток. = 1,1 * 12 = 13,6 месяцев

5. Расчет коэффициента экономической эффективности:

Коэффициент экономической эффективности показывает, сколько рублей годовой экономии приносит каждый рубль, вложенный в разработку:

$$E = Э / Робщ. \qquad (2.30)$$

Подставляем значения:

E = 1 041 978 / 1 180 511 = 0,9

Нормативный коэффициент эффективности при сроке окупаемости 3 года составляет:

Ен = 1 / Тн = 1 / 3 = 0,33

Сравнение фактического коэффициента с нормативным:

0,9 > 0,33 – условие выполняется.

Проведенные расчеты позволяют сделать следующие выводы:

- Годовые затраты на работу без внедрения ядра цифровой экосистемы составляют 1 267 200 рублей. После внедрения разработанной платформы годовые затраты снижаются до 225 222 рублей. Годовой экономический эффект от использования программного продукта составляет 1 041 978 рублей;
- Срок окупаемости затрат на разработку и внедрение программного продукта равен 13,6 месяцам, что значительно меньше нормативного срока окупаемости в 36 месяцев (3 года);
- Коэффициент экономической эффективности составляет 0,9, что превышает нормативное значение 0,33.

Таким образом, разработка и внедрение ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом является экономически целесообразным. Высокая эффективность проекта обусловлена значительной экономией рабочего времени специалистов при переходе от старой архитектуры к микросервисной.

Охрана труда

Общие требования охраны труда:

1. К работе программистом допускаются лица не моложе 18 лет, имеющие соответствующую выполняемой работе квалификацию, прошедшие вводный и первичный на рабочем месте инструктажи по охране труда, медосмотр, обучение и проверку знаний по охране труда.

2. Для выполнения работ на персональном компьютере программист должен изучить инструкцию по эксплуатации персонального компьютера, на котором работник выполняет работы, пройти инструктаж по электробезопасности и получить I группу.

3. Программист, выполняющий работу на персональном компьютере, независимо от квалификации и стажа работы не реже одного раза в шесть месяцев должен проходить повторный инструктаж по безопасности труда. в случае нарушения требований безопасности труда, при перерыве в работе более чем на 60 календарных дней программист должен пройти внеплановый инструктаж.

4. Программист, показавший неудовлетворительные навыки и знания требований безопасности при работе на персональном компьютере, к самостоятельной работе не допускается.

5. Программист, допущенный к самостоятельной работе, должен знать:

5.1. Правила эксплуатации и требования безопасности при работе с персональным компьютером.

5.2. Способы рациональной организации рабочего места.

5.3. Санитарно-гигиенические требования к условиям труда.

5.4. Опасные и вредные производственные факторы, которые могут оказывать неблагоприятное воздействие на программиста.

6. Программист, направленный для участия в несвойственных его профессии работах, должен пройти целевой инструктаж по безопасному выполнению предстоящих работ.

7. Программист, работающий на персональном компьютере, должен соблюдать установленные для него режимы труда и отдыха.

8. Для предупреждения возможности возникновения пожара программист должен соблюдать требования пожарной безопасности сам и не допускать нарушений со стороны других работников.

9. Для предупреждения заболеваний программисту следует знать и соблюдать правила личной гигиены.

10. В случае заболевания, плохого самочувствия, недостаточного отдыха программисту следует сообщить о своем состоянии непосредственному руководителю и обратиться за медицинской помощью.

11. Если программист оказался очевидцем несчастного случая, он должен оказать пострадавшему первую помощь и сообщить о случившемся руководителю.

12. Программист должен уметь оказать первую помощь, в том числе при поражении электрическим током, пользоваться аптечкой.

13. Программист, допустивший нарушение или невыполнение требований инструкции по охране труда, несет ответственность согласно действующему законодательству.

Опасные и вредные производственные факторы:

1. Перенапряжение зрительного анализатора при работе за экраном дисплея.

2. Длительное статическое напряжение мышц спины, шеи, рук и ног, что может привести к статическим перегрузкам программиста.

3. Повышенный уровень шума.

4. Ионизирующие и неионизирующие излучения, источниками которых являются видеодисплейные терминалы.

5. Статическое электричество.

6. Электрический ток, путь которого в случае замыкания на корпус может пройти через тело человека.

Рабочие помещения и рабочие места оператора ПК должны соответствовать определенным требованиям, которые обеспечат комфортную и безопасную работу. К ним относятся:

1. Размеры помещения не менее 6 квадратных метров на одно рабочее место.

2. Наличие естественного и искусственного освещения, соответствующего СанПиН 2.2.2./2.4.134003 «Гигиенические требования к условиям труда при работе на персональных компьютерах».

3. Температура воздуха в помещении должна находиться в пределах 1824 градусов Цельсия, влажность 40-60%.

4. Отсутствие шума и других нежелательных звуковых эффектов.

5. Наличие пожарных средств и путей эвакуации.

6. Наличие мебели, соответствующей правилам эргономики (регулируемая высота, регулируемый наклон стола, спинка и сиденье стула, наличие подставки для ног и т.д.).

7. Наличие специального оборудования для защиты здоровья (экраны, фильтры, подставки для документов и т.д.).

8. Соответствие параметров компьютера установленным нормам.

9. Работа оператора ПК должна проходить в условиях микроклимата, обеспечивающего наиболее комфортные условия для зрения (уровень освещенности должен быть не менее 300 лк).

10. Обеспечение необходимой пространственной ориентации с помощью различных методов размещения монитора и клавиатуры.

Общие требования при использовании компьютерной техники:

1. Защита зрения.

1.1. Правильное расстояние до экрана (60-70 см), регулировка яркости и контрастности, отсутствие бликов.

1.2. Использование щадящих цветовых схем (светло-зеленые, желто-зеленые тона), темные символы на светлом фоне.

1.3. Каждые 20-30 минут отводить взгляд от экрана, фокусироваться на удаленных объектах (правило “202020”: каждые 20 минут смотреть на объект на расстоянии 20 футов (около 6 метров) в течение 20 секунд).

1.4. Выполнение простых упражнений для расслабления мышц глаз (например, перемещение взгляда вверх-вниз, вправо-влево, по кругу).

1.5. Использование увлажняющих капель (“искусственная слеза”) при сухости глаз.

1.6. Правильное освещение рабочего места (не слишком яркое, не слишком тусклое, без бликов), использование настольной лампы.

1.7. Использование программ, фильтрующих синий свет, изменяющих цветовую температуру экрана в зависимости от времени суток.

1.8. Регулярные осмотры у офтальмолога (не реже одного раза в год).

1.9. Использование мониторов с матовым покрытием или антибликовыми пленками.

1.10. Настройка параметров шрифта для комфортного чтения.

2. Защита органов слуха.

2.1. Использование тихо работающих компьютеров и периферийных устройств (вентиляторов, жестких дисков).

2.2. Обеспечение звукоизоляции рабочего места (например, использование звукопоглощающих панелей).

2.3. Использование наушников или гарнитуры для телефонных разговоров, что позволяет снизить уровень окружающего шума. При этом громкость звука в наушниках должна быть умеренной.

2.4. Регулярные перерывы в тишине, отдых от звуковых раздражителей.

2.5. Использование берушей при работе в шумной обстановке.

2.6. Регулярные осмотры у отоларинголога (особенно при наличии факторов риска, таких как работа в шумной среде).

3. Защита органов пищеварения.

3.1. Соблюдение режима питания (завтрак, обед, ужин), прием пищи в одно и то же время.

3.2. Употребление достаточного количества овощей, фруктов, цельнозерновых продуктов, нежирного белка. Ограничение потребления жирной, жареной, сладкой пищи, фастфуда.

3.3. Употребление достаточного количества воды (не менее 1,52 литров в день).

3.4. Выбор полезных перекусов (фрукты, овощи, орехи, йогурт).

3.5. Использование обеденного перерыва для полноценного приема пищи, отдыха от работы.

3.6. Рекомендуется есть в специально отведенных для этого местах.

3.7. Соблюдать режим питания, не переедать перед сном.

3.8. Тщательно мыть руки перед едой.

4. Защита кожных покровов.

4.1. Использование увлажнителей воздуха в помещении.

4.2. Регулярное проветривание помещения.

4.3. Умывание лица несколько раз в день.

4.4. Использование увлажняющих кремов для рук и лица.

4.5. Использование минимального количества косметики, выбор гипоаллергенной косметики.

4.6. Хотя прямое воздействие электромагнитного излучения компьютеров на кожу незначительно, рекомендуется соблюдать дистанцию от экрана и других устройств.

4.7. Регулярная уборка рабочего места, очистка клавиатуры и мыши от пыли и грязи.

4.8. Использование антистатических ковриков.

5. Установка расписания работы и время отдыха в течение дня.

5.1. Соблюдение режима труда и отдыха, установленного нормативными документами. Продолжительность непрерывной работы с видеомонитором не должна превышать 2 часов.

5.2. Короткие перерывы (1-3 минуты) каждые 20-30 минут для снятия напряжения.

5.3. Выполнение простых физических упражнений во время перерывов для улучшения кровообращения и снятия мышечного напряжения.

5.4. Чередование различных видов деятельности, планирование задач с учетом пиков и спадов работоспособности.

5.5. Достаточный сон (78 часов) для восстановления сил.

5.6. Полноценный отдых в выходные дни.

5.7. Регулярный отпуск для восстановления физического и психического здоровья.

5.8. Правильная организация рабочего места (удобное кресло, регулируемый стол, правильное расположение монитора и клавиатуры) помогает снизить утомляемость и риск возникновения заболеваний.

Техника безопасности перед началом работы:

1. Перед началом работы программисту следует рационально организовать свое рабочее место.

2. Программист должен знать о том, что если в помещении расположены несколько персональных компьютеров, то для обеспечения безопасности расстояние между ними должно быть не менее 1,5 м.

3. Программист должен знать о том, что взаимное расположение персональных компьютеров влияет на уровень генерируемых ими излучений. для предупреждения облучения других рабочих мест следует выполнять следующие правила:

1.1. Левая панель персонального компьютера должна быть обращена либо к стене, либо к проходу, где нет рабочих мест.

1.2. Не следует располагать мониторы экранами друг к другу.

1.3. Не рекомендуется располагать монитор экраном к окну.

1.4. Для того, чтобы в процессе работы не возникало перенапряжение зрительного анализатора, программисту следует

проверить, чтобы на клавиатуре и экране монитора не было бликов света.

1.5. Для повышения контрастности изображения перед началом работы программист должен очистить экран монитора от пыли, которая интенсивно оседает на нем под воздействием зарядов статического электричества.

4. Программист должен убрать с рабочего места все лишние предметы, не используемые в работе.

5. Перед включением персонального компьютера программисту следует визуально проверить исправность электропроводки, вилки, розетки, а также электрических подсоединений между собой всех устройств, входящих в комплект персонального компьютера.

6. Перед началом выполнения работы программист должен проверить исправность персонального компьютера и подготовить его к работе.

Техника безопасности во время работы

1. Программисту персонального компьютера следует включать его в работу в той последовательности, которая определена инструкцией по эксплуатации.

2. Для подключения персонального компьютера к электрической сети программист должен использовать шнур питания, поставляемый в комплекте с персональным компьютером. не следует использовать самодельные электрические шнуры для подключения к сети персонального компьютера и различных его устройств.

3. Программист должен знать, что рациональная рабочая поза способствует уменьшению утомляемости.

4. При помощи поворотной площадки видеомонитор должен быть отрегулирован в соответствии с рабочей позой программиста.

5. Конструкция рабочего стула (кресла) должна обеспечивать поддержание рабочей позы программиста при работе с персональным

компьютером, позволять изменять позу с целью снижения статического напряжения мышц шейно-плечевой области и спины для предупреждения развития утомления.

6. Тип рабочего стула (кресла) должен выбираться в зависимости от характера и продолжительности работы с персональным компьютером с учетом роста программиста.

7. Рабочий стул (кресло) должен быть подъемно-поворотным и регулируемым по высоте и углам наклона сиденья и спинки, а также расстоянию спинки от переднего края сиденья. при этом регулировка каждого параметра должна быть независимой, легко осуществляемой и иметь надежную фиксацию.

8. Поверхность сиденья, спинки и других элементов стула (кресла) должна быть полумягкой, с нескользящим, неэлектризуемым и воздухопроницаемым покрытием, обеспечивающим легкую очистку от загрязнений.

9. Плоскость рабочего стола должна быть регулируемой по высоте в пределах 680–800 мм с учетом индивидуальных особенностей программиста. при отсутствии такой возможности высота рабочей поверхности стола должна составлять 725 мм.

10. Рабочий стол должен иметь пространство для ног высотой не менее 600 мм, шириной – не менее 500 мм, глубиной на уровне колен – не менее 450 мм и на уровне вытянутых ног – не менее 650 мм.

11. Экран видеомонитора должен находиться от глаз программиста на оптимальном расстоянии 600–700 мм, но не ближе 500 мм с учетом размеров алфавитно-цифровых знаков и символов.

12. Клавиатуру следует располагать на поверхности стола на расстоянии 100–300 мм от края, обращенного к пользователю, или на специальной, регулируемой по высоте рабочей поверхности, отделенной от основной столешницы.

13. Для уменьшения зрительной утомляемости программисту предпочтительнее работать в таком режиме, чтобы на светлом экране видеомонитора были темные символы.

14. С целью снижения зрительного и костно-мышечного утомления программисту следует соблюдать установленный режим труда и отдыха.

15. Режимы труда и отдыха при работе с персональным компьютером должны организовываться в зависимости от вида и категории трудовой деятельности.

Требования охраны труда в аварийных ситуациях

1. При обнаружении каких-либо неполадок в работе персонального компьютера программист должен прекратить работу, выключить компьютер и сообщить об этом непосредственному руководителю для организации ремонта.

2. Программисту не следует самому устранять технические неполадки персонального компьютера.

3. Программист не должен производить работу при снятом корпусе компьютера.

4. При несчастном случае, отравлении, внезапном заболевании необходимо немедленно оказать первую помощь пострадавшему, вызвать врача или помочь доставить пострадавшего к врачу, а затем сообщить руководителю о случившемся.

5. Программист должен уметь оказывать первую помощь при ранениях. при этом он должен знать, что всякая рана легко может загрязниться микробами, находящимися на ранящем предмете, коже пострадавшего, а также в пыли, на руках оказывающего помощь и на грязном перевязочном материале.

6. При обнаружении пожара или признаков горения (задымление, запах гари, повышение температуры и т. п.) необходимо немедленно уведомить об этом пожарную охрану по телефону 01.

7. До прибытия пожарной охраны нужно принять меры по эвакуации людей, имущества и приступить к тушению пожара.

8. При возгорании персонального компьютера программист должен отключить его от источника тока и приступить к тушению своими силами. при этом следует помнить, что для тушения установок, находящихся под напряжением, применяют углекислотные или порошковые огнетушители.

Техника безопасности по окончании работы

1. По окончании работы программист должен выключить персональный компьютер и отсоединить сетевой шнур от электрической сети.

2. Программист должен привести в порядок рабочее место, убрать документацию и т. п.

Современная жизнь невозможна без использования компьютеров и интернета. Тем не менее, работа на ПК может представлять опасность для здоровья человека, если не соблюдать правила безопасности. Такие заболевания, как синдром карпального канала, синдром "сухого глаза", головные боли и шум в ушах, связаны с длительным пребыванием за компьютером. В связи с этим, необходимо соблюдать следующие правила техники безопасности при работе на ПК:

1. Регулярно делать перерывы. При работе за компьютером необходимо делать перерыв каждые 45-60 минут. Во время перерывов рекомендуется выполнять упражнения для глаз и рук.

2. Правильно настроить рабочее место. Рабочее место должно быть правильно настроено, чтобы минимизировать риск различных заболеваний. Клавиатура должна быть расположена на уровне локтя, а экран на уровне глаз.

3. Использовать эргономическую мебель. Для работы за компьютером рекомендуется использовать эргономические кресла и столы, которые помогают поддерживать правильную позу.

4. Соблюдать правильный режим работы. Необходимо соблюдать правильный режим работы и сна, чтобы избежать усталости и стресса, которые могут привести к заболеваниям.

5. Использовать программы для защиты глаз. Существует множество программ, которые помогают защитить глаза при работе за компьютером. Они уменьшают яркость экрана, фильтруют синий свет и т.д.

6. Использовать антивирусное программное обеспечение. Антивирусное программное обеспечение помогает защитить компьютер от вирусов и злонамеренных программ, которые могут повредить систему или украсть личную информацию.

7. Избегать монотонной нагрузки. Повторяющиеся действия могут привести к снижению производительности и здоровью. Для этого рекомендуется использовать различные программы и ресурсы, менять виды деятельности, чтобы предотвратить нагрузку на конкретную группу мышц.

8. Использовать безопасные пароли. Для защиты личной информации и конфиденциальных данных следует использовать сложные пароли, которые не легко угадать или взломать. Рекомендуется использовать комбинации букв, цифр и символов.

9. Создавать резервные копии данных. Для сохранения важной информации необходимо регулярно создавать резервные копии данных, чтобы предотвратить потерю или повреждение важных файлов.

10. Обновлять программное обеспечение. Регулярное обновление программного обеспечения помогает устранять уязвимости и предотвращать атаки злонамеренных программ. Рекомендуется устанавливать обновления операционной системы, браузера и антивирусного ПО.

11. Использовать безопасную сеть. При работе на публичных сетях необходимо использовать VPN для защиты информации, передаваемой между компьютером и сервером или другим устройством.

12. Избегать использования нелицензионного программного обеспечения. Использование нелицензионного программного обеспечения может привести к нарушению законодательства и повышенному риску заражения вирусами и злонамеренными программами.

В целом, соблюдение правил техники безопасности при работе на ПК имеет важное значение для сохранения здоровья и безопасности человека. Рекомендуется следовать простым правилам, чтобы избежать проблем, связанных с длительным пребыванием за компьютером.

Заключение

Список используемых источников

Приложение А

(обязательное)

Графическая часть

Приложение Б

(обязательное)

Техническое задание

1 Введение

Полное наименование темы дипломного проекта – Разработка ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом.

Разработчик программного продукта – студент группы 81-2022 Газзаев Александр.

Разрабатываемое программное обеспечение предназначено для создания ядра цифровой экосистемы на основе микросервисной архитектуры, обеспечивающего централизованную аутентификацию пользователей, маршрутизацию запросов через API-шлюз и взаимодействие сервисов в единой инфраструктуре.

Для достижения поставленной цели сформированы следующие задачи:

1. анализ существующих решений и архитектур цифровых экосистем;
2. проектирование архитектуры ядра цифровой экосистемы с учетом принципов масштабируемости, отказоустойчивости и безопасности;
3. разработка и реализация основных компонентов системы;
4. тестирование разработанного программного обеспечения;
5. подготовка технической и эксплуатационной документации на разработанную систему.

2 Общие сведения

Основанием для разработки является дипломное задание на выпускную квалификационную работу по специальности «Информационные системы и программирование».

Актуальность темы заключается в том, что в условиях стремительной цифровизации и роста числа онлайн-сервисов возрастает потребность в построении масштабируемых и безопасных цифровых экосистем. Современные компании стремятся объединить различные сервисы в единую платформу с централизованным управлением доступом, единым механизмом аутентификации и унифицированным способом взаимодействия между компонентами системы. Особенно остро эта задача проявляется при увеличении количества пользователей, сервисов и объёма обрабатываемых данных.

При отсутствии архитектурно выстроенного ядра цифровой экосистемы возникают проблемы дублирования функциональности, усложняется сопровождение сервисов, снижается уровень безопасности и управляемости инфраструктуры. Разрозненные механизмы аутентификации и прямое взаимодействие сервисов без API-шлюза приводят к повышенным рискам утечки данных, усложняют масштабирование и интеграцию новых компонентов.

Отсутствие централизованной системы управления доступом и маршрутизации запросов увеличивает нагрузку на разработчиков и администраторов, усложняет мониторинг и контроль, а также снижает устойчивость системы к сбоям. Внедрение ядра цифровой экосистемы на основе микросервисной архитектуры с единой аутентификацией и API-шлюзом позволяет обеспечить гибкость, отказоустойчивость, безопасность и удобство дальнейшего расширения платформы, что делает разработку данного решения актуальной и востребованной задачей.

3 Назначение и цели разработки

Разрабатываемое программное обеспечение не ориентировано на конкретную организацию и предназначено для использования в качестве базовой платформы (ядра) цифровой экосистемы, обеспечивающей интеграцию различных сервисов в единую инфраструктуру.

Назначение системы заключается в создании универсального технологического решения, реализующего микросервисную архитектуру с централизованной аутентификацией, авторизацией и маршрутизацией запросов через API-шлюз. Разрабатываемое ядро цифровой экосистемы обеспечивает единый механизм управления доступом пользователей, безопасное взаимодействие сервисов, масштабируемость и возможность подключения новых модулей без существенной переработки существующей архитектуры.

Внедрение такого решения позволит организациям формировать собственные цифровые экосистемы на базе единой платформы, снизить сложность интеграции сервисов, повысить уровень информационной безопасности и упростить сопровождение программной инфраструктуры.

3.1 Функциональное назначение

Целью разработки является создание ядра цифровой экосистемы, обеспечивающего:

- централизованную аутентификацию и авторизацию пользователей;
- маршрутизацию клиентских запросов через API-шлюз к соответствующим микросервисам;
- безопасное взаимодействие между сервисами;
- управление правами доступа на основе ролей и политик безопасности;
- возможность масштабирования и расширения системы за счет добавления новых микросервисов.

4 Требования к программе или программному изделию

4.1 Требования к характеристикам

4.1.1 Требования к составу выполняемых функций

Функциональные требования определяют основные возможности системы и реализуемые ею функции:

1. Система должна обеспечивать централизованную аутентификацию пользователей.
2. Система должна обеспечивать авторизацию пользователей на основе ролей и политик доступа.
3. Система должна предоставлять API-шлюз в качестве единой точки входа для маршрутизации внешних клиентских запросов к соответствующим микросервисам.
4. API-шлюз должен обеспечивать проверку токенов доступа и фильтрацию неавторизованных запросов.
5. Система должна обеспечивать регистрацию и управление микросервисами в единой инфраструктуре, включая их обнаружение.
6. Система должна обеспечивать безопасное межсервисное взаимодействие.
7. Система должна поддерживать механизм выдачи, обновления и отзыва токенов доступа.
8. Администратор должен иметь возможность управления ролями и правами доступа пользователей.
9. Администратор должен иметь возможность регистрации, блокировки и удаления учетных записей пользователей.
10. Система должна обеспечивать логирование действий пользователей и сервисов.
11. Система должна обеспечивать мониторинг состояния микросервисов.
12. Система должна обеспечивать централизованную обработку ошибок и формирование унифицированных ответов API.

Нефункциональные требования определяют характеристики качества системы, ограничения и условия её функционирования:

1. Система должна поддерживать масштабирование сервисов без нарушения общей архитектуры и работоспособности системы.
2. Система должна обеспечивать защиту от несанкционированного доступа, включая механизмы ограничения частоты запросов (rate limiting).
3. Система должна обеспечивать возможность интеграции новых микросервисов без модификации ядра системы за счёт слабой связанности компонентов.
4. Система должна обеспечивать отказоустойчивость и стабильную работу при частичных сбоях отдельных сервисов.
5. Система должна обеспечивать высокую производительность при обработке пользовательских запросов.
6. Система должна обеспечивать безопасность хранения и передачи данных (шифрование, защита токенов и учетных данных).
7. Система должна быть удобной для сопровождения и расширения за счёт применения модульной архитектуры и принципов разделения ответственности.

4.1.2 Требования к организации входных и выходных данных

Входными данными системы являются:

- учетные данные пользователей (логин, пароль и иные идентификационные сведения);
- токены доступа;
- HTTP-запросы от клиентских приложений;
- служебные данные, передаваемые между микросервисами.

Входные данные могут относиться к конфиденциальной информации и должны передаваться по защищённым каналам связи с использованием современных протоколов шифрования.

Выходные данные системы должны быть организованы в формате унифицированных ответов API (например, в формате JSON), содержащих результаты обработки запросов, коды состояния и сообщения об ошибках.

4.1.3 Требования к временным характеристикам

После изменения данных, находящихся в базе данных, новая информация отображается не позднее, чем через 2 секунды.

4.2 Требования к надежности

4.2.1 Требования к обеспечению надежного (устойчивого) функционирования программы

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением совокупности организационно-технических мероприятий, перечень которых приведен ниже:

а) исправностью оборудования и наличием необходимых характеристик технических и программных средств;

б) приложение не должно аварийно завершаться при некорректных действиях пользователя (контроль входных данных);

в) необходимым уровнем квалификации пользователя.

4.2.2 Время восстановления после отказа

Время восстановления после отказа, вызванного сбоем электропитания технических средств (иными внешними факторами), не фатальным сбоем (не крахом) операционной системы, не должно превышать 10 минут при условии соблюдения условий эксплуатации технических и программных средств.

Время восстановления после отказа, вызванного неисправностью технических средств, фатальным сбоем (крахом) операционной системы, не должно превышать времени, требуемого на устранение неисправностей технических средств и переустановки программных средств.

4.2.3 Отказы из-за некорректных действий оператора

Отказы программы возможны вследствие некорректных действий оператора (пользователя) при взаимодействии с операционной системой. Во избежание возникновения отказов программы по указанной выше причине следует обеспечить работу пользователя без предоставления ему административных привилегий.

4.3 Условия эксплуатации

4.3.1 Климатические условия эксплуатации

Специальные условия не требуются.

4.3.2 Требования к видам обслуживания

Программа не требует проведения каких-либо видов обслуживания.

4.4 Требования к составу и параметрам технических средств

Минимальные требования:

- Windows 11*

Примечание. На устройствах под управлением Windows 11 должны быть установлены версии Windows 11 Домашняя, Профессиональная или Корпоративная. Режим S не поддерживается.

Параметры и состав технических средств сервера определяются требованиями операционных систем и ПО, необходимых для нормальной работы приложения.

4.5 Требования к маркировке и упаковке

Дополнительных требований к маркировке и упаковке не предъявляется.

4.6 Требования к транспортированию и хранению

Требования к транспортированию и хранению не предъявляются.

4.7 Специальные требования

Приложение должно обеспечивать взаимодействие с пользователем посредством графического пользовательского интерфейса.

5 Требования к программной документации

5.1 Предварительный состав программной документации

Состав программной документации должен включать в себя:

- техническое задание;
- руководство пользователя;
- листинг программы;
- руководство системного программиста
- пояснительная записка

Специальные требования к программной документации не предъявляются.

6 Стадии и этапы разработки

Разработка должна быть проведена в три стадии:

1. техническое задание;

2. технический (и рабочий) проекты;

3. внедрение.

На стадии «Техническое задание» должен быть выполнен этап разработки, согласования и утверждения настоящего технического задания.

На стадии «Технический (и рабочий) проект» должны быть выполнены перечисленные ниже этапы работ:

– разработка программы;

– разработка программной документации;

– испытания программы.

На стадии «Внедрение» должен быть выполнен этап разработки «Подготовка и передача программы».

Содержание работ по этапам:

На этапе разработки технического задания должны быть выполнены перечисленные ниже работы:

– постановка задачи;

– определение и уточнение требований к техническим средствам;

– определение требований к программе;

– определение стадий, этапов и сроков разработки программы и документации на нее;

– согласование и утверждение технического задания.

На этапе разработки программы должна быть выполнена работа по программированию (кодированию) и отладке программы.

На этапе разработки программной документации должна быть выполнена разработка программных документов в соответствии с требованиями ГОСТ 19.101-77.

На этапе испытаний программы должны быть выполнены перечисленные ниже виды работ:

– разработка, согласование и утверждение порядка и методики испытаний;

– проведение приемо-сдаточных испытаний;

– корректировка программы и программной документации по результатам испытаний.

На этапе подготовки и передачи программы должна быть выполнена работа по подготовке и передаче программы и программной документации в эксплуатацию на объектах заказчика.

7 Порядок контроля и приемки

Приемосдаточные испытания программы должны проводиться согласно разработанной исполнителем и согласованной заказчиком «Программы и методики испытаний».

Ход проведения приемо-сдаточных испытаний заказчик и исполнитель документируют в протоколе испытаний.

На основании протокола испытаний исполнитель совместно с заказчиком подписывают акт приемки-сдачи программы в эксплуатацию.

Приложение В

(обязательное)

Руководство системного программиста

Приложение Г

(обязательное)

Руководство пользователя

Приложение Д

(обязательное)

Фрагмент листинга программы